
Relazione - Game Handler

Progetto del corso di PISSIR - 2025/26

Autore

Emanuele Trento, Davide
Castellani, Tiziano Ceccon

Categoria

Progetto del corso di PISSIR

Data

07/19/2026

Contents

GAME HANDLER **2**

1. DESCRIZIONE DEL PROGETTO 2

2. STRUTTURA DEL SISTEMA 3

3. GIOCHI, SENSORI E INTERFACCIA LOCALE 3

4. TORNEI, STATISTICHE E INTERFACCIA UTENTE 3

5. FASI DI LAVORO 8

GAME HANDLER

Progetto del corso di PISSIR, anno 2025/26

1. DESCRIZIONE DEL PROGETTO

Obbiettivo

L'obiettivo del progetto è la realizzazione di una piattaforma distribuita per la gestione di giochi da tavolo e da bar, che permetta la creazione di tornei e di raccogliere e analizzare dati sullo svolgimento delle partite tramite l'utilizzo di sensori.

Questo avviene perchè ogni gioco ha dei sensori che rilevano gli input significativi (i gol in un calcetto ad esempio), questi vengono convertiti dalla board sul gioco (ad esempio un ESP32) in chiamate http e mqtt al broker o al server locale. Prima di poter giocare è necessario autenticarsi o creare un utente.

Utenti di sistema

Ci sono quattro tipi di utenti:

- 1) PLAYER -> il giocatore
- 2) LOCAL_ADMIN -> l'admin del locale (il bar)
- 3) GAME_ADMIN -> l'admin che gestisce i giochi
- 4) PLATFORM_ADMIN -> il super admin

La tabella descrive le funzionalità di ogni utente

Permesso	PLAYER	LOCAL_ADMIN	GAME_ADMIN	PLATFORM_ADMIN
Partecipa alle partite	✓			✓
Consulta proprie statistiche	✓			✓
Visualizza i giochi disponibili nei locali	✓	✓		✓
Partecipa ai tornei	✓			✓
Gestisce i giochi presenti nel proprio locale		✓		✓
Configura i dispositivi e monitora le partite		✓		✓
Visualizza statistiche relative al locale		✓		✓
Definisce nuove tipologie di giochi			✓	✓
Configura le regole di registrazione delle partite			✓	✓
Gestisce utenti e locali				✓
Monitora il funzionamento dell'intero sistema				✓
Accede a statistiche globali				✓
Creazione dei tornei				✓

Maggiori informazioni in [ruoli_utenti](#)

2. STRUTTURA DEL SISTEMA

E' composto da tre componenti:

- **Central System (L'Hub):** Microservizio Spring Boot responsabile della *Source of Truth* globale: registrazione utenti, aggregazione statistiche cross-building, coordinamento sincronizzazione.
- **Local Server (Lo Spoke / Edge Node):** Microservizio Spring Boot installato fisicamente in ogni edificio. Funziona come gateway e nodo di persistenza locale. Persiste gli stati dei giochi nel database locale e genera le statistiche localmente (partite completate, in corso, tempo di utilizzo). I dati aggregati delle statistiche vengono poi inviati al Central System.
- **Endpoint (Game Clients):** Applicazioni client con interfaccia grafica (JavaFX) che comunicano **esclusivamente** con il Local Server del proprio edificio tramite MQTT over TLS. L'uso di MQTT disaccoppia i client dal server ed è compatibile con la futura integrazione ESP32/Arduino.

Per la comunicazione tra le varie parti ci affidiamo sia ad API REST (usate ad esempio per gli accessi) che al protocollo MQTT.

Maggiori informazioni sulla comunicazione tra le parti in [messages_flow](#)

Maggiori informazioni sull'architettura in [architettura_proposta](#) e [architettura_classi](#)

Funzionamento offline

Alla creazione di un utente nuovo, i dati vengono salvati in un database locale e creato un evento nell'outbox. Ogni transizione (start, pause, resume e end) avviene nel db locale, gli eventi vengono poi pubblicati tramite MQTT sul broker locale, alla fine della sessione viene pubblicato un evento GAME_SESSION_COMPLETE. Ragionamento simile è quello per le prenotazioni, vengono infatti create e salvate sul db locale e ogni azione genera un evento. In caso di crash del Local Server, il server recupera le sessioni attive o in pausa dal DB, fa un ping delle macchine tramite MQTT locale e se entro 30 secondi non riceve risposta chiude il gioco con un evento GAME_SESSION_COMPLETE.

In pratica quando è offline il sistema usa solo il server locale normalmente registrando tutto ciò che accade. Ogni cinque minuti fa un ping al Central System, se lo rileva online, il locale invia tutti gli eventi pending in un unico payload e svuota la coda. A questo punto il server centrale processa gli eventi, aggiorna i dati (statistiche, utenti e prenotazioni).

Maggiori informazioni [local_offline_handling](#)

3. GIOCHI, SENSORI E INTERFACCIA LOCALE

Ogni gioco avrà dei sensori specifici per raccogliere le informazioni sull'andamento delle partite, non avendo giochi fisici non abbiamo deciso il posizionamento sui singoli giochi dei sensori ma ne abbiamo simulati alcuni come scacchi o slot machine.

4. TORNEI, STATISTICHE E INTERFACCIA UTENTE

I tornei possono essere organizzati dagli utenti PLATFORM_ADMIN e sono ad eliminazione diretta. Sono organizzati su almeno due edifici diversi e sullo stesso gioco. Esistono due varianti:

- **Individuale** -> usa l'userId del giocatore
- **A squadre** -> viene registrato un team con un teamId e la lista dei giocatori del team

Game Client Emulator

GamesMy StatsMy MatchesTournamentsAggregated StatsLocal DashboardGame AdminPlatform AdminAdmin RequestsLogout

Create tournament (POST)

tournamentId

OpenCancelSchedule

newName

startsAt ISO-8601 (e.g. 2030-01-01T00

buildingIds (comma-separated, ≥2)

Update (PUT) DRAFT-only

Delete (DELETE) DRAFT-only

Standings & bracket — see "Tournaments" (variable)
(reusing PLAYER views)

Global statistics (GET /api/statistics)

Load global statistics

Local-server monitoring (GET /api/admin/servers/health)

buildingId	baseUrl	lastSeenAt	active	pendingReplica	Action
building-1	https://local-server-1:8181	2026-07-17T20:07:03Z	true	0	Toggle active

My building: building-1 active=true pending=1

Connected as: platformadmin1 · roles=[PLATFORM_ADMIN]

Game Client Emulator

GamesMy StatsMy MatchesTournamentsAggregated StatsLocal DashboardGame AdminPlatform AdminAdmin RequestsLogout

Tournaments

Refresh tournamentsRegister me (self)Register teamMy matches / StartStart selected match

Test Tour [SLOT_MACHINE] — COMPLETED starts 2026-07

Standings

Bracket (matches)

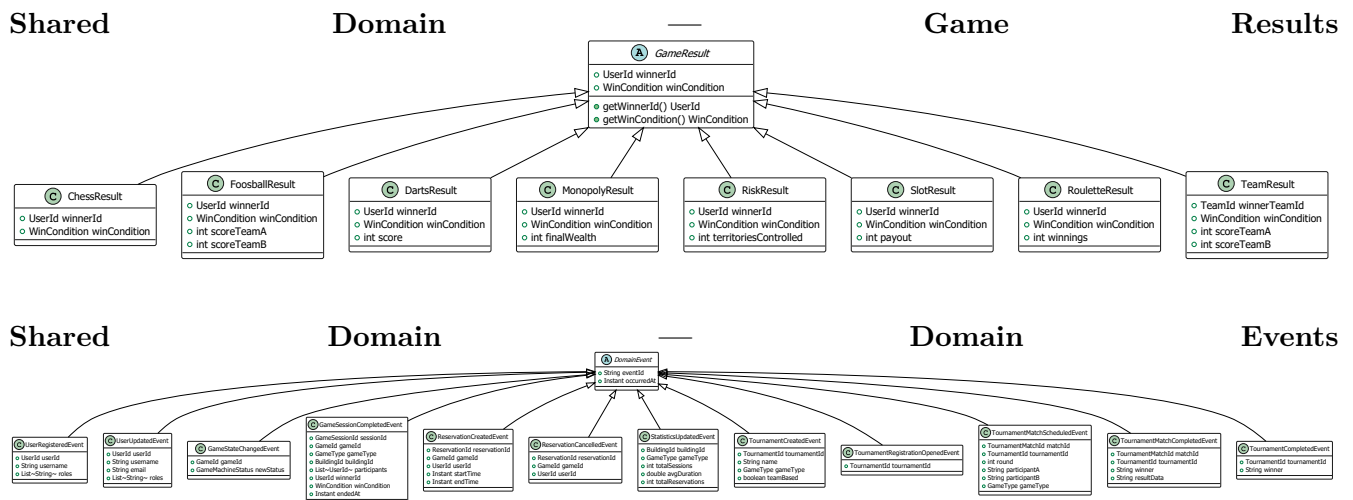
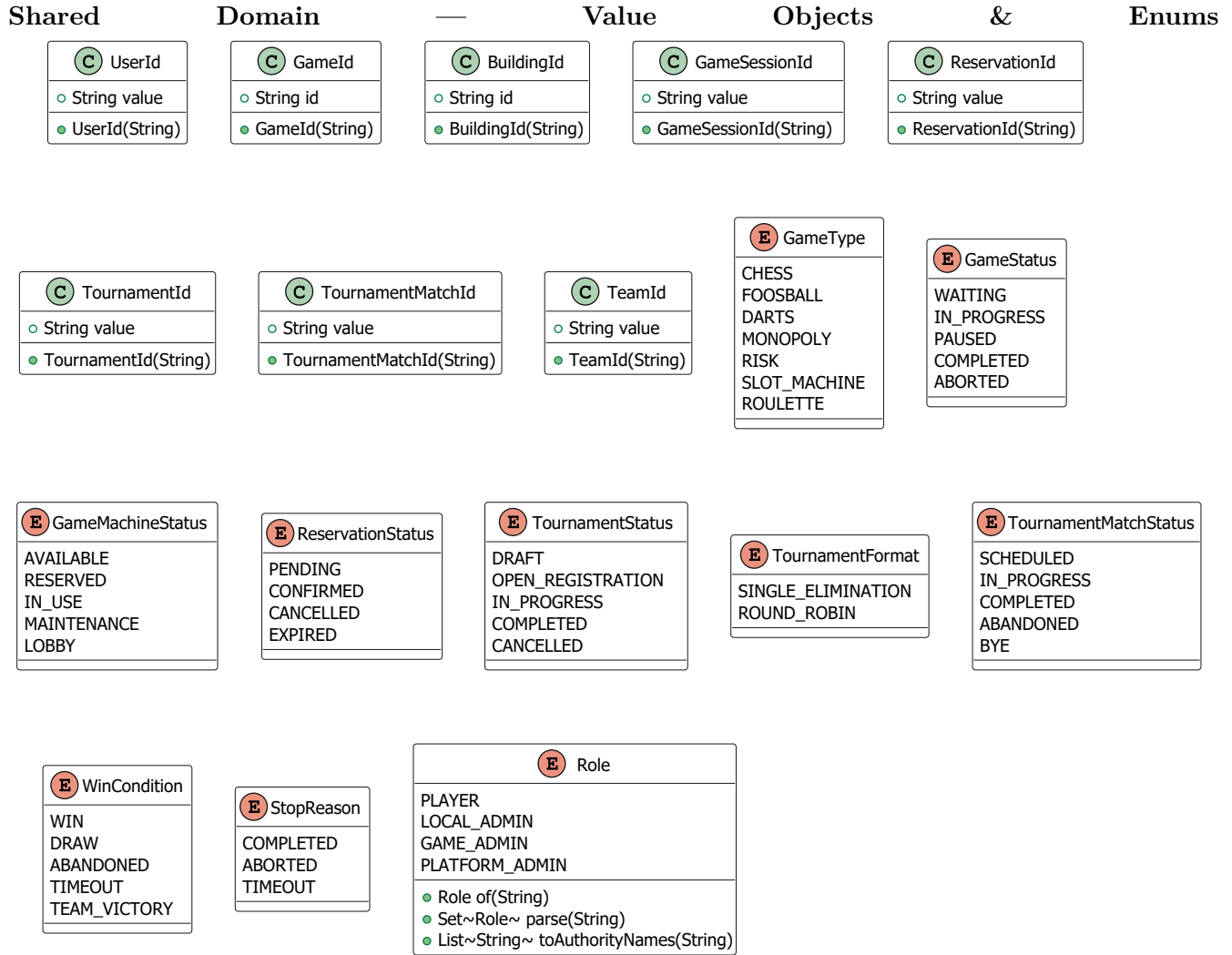
Participants

My matches (SCHEDULED)

1 tournaments Data updated at: 10:01:58 waiting for replication

Connected as: platformadmin1 · roles=[PLATFORM_ADMIN]

Diagramma UML delle Classi del Dominio



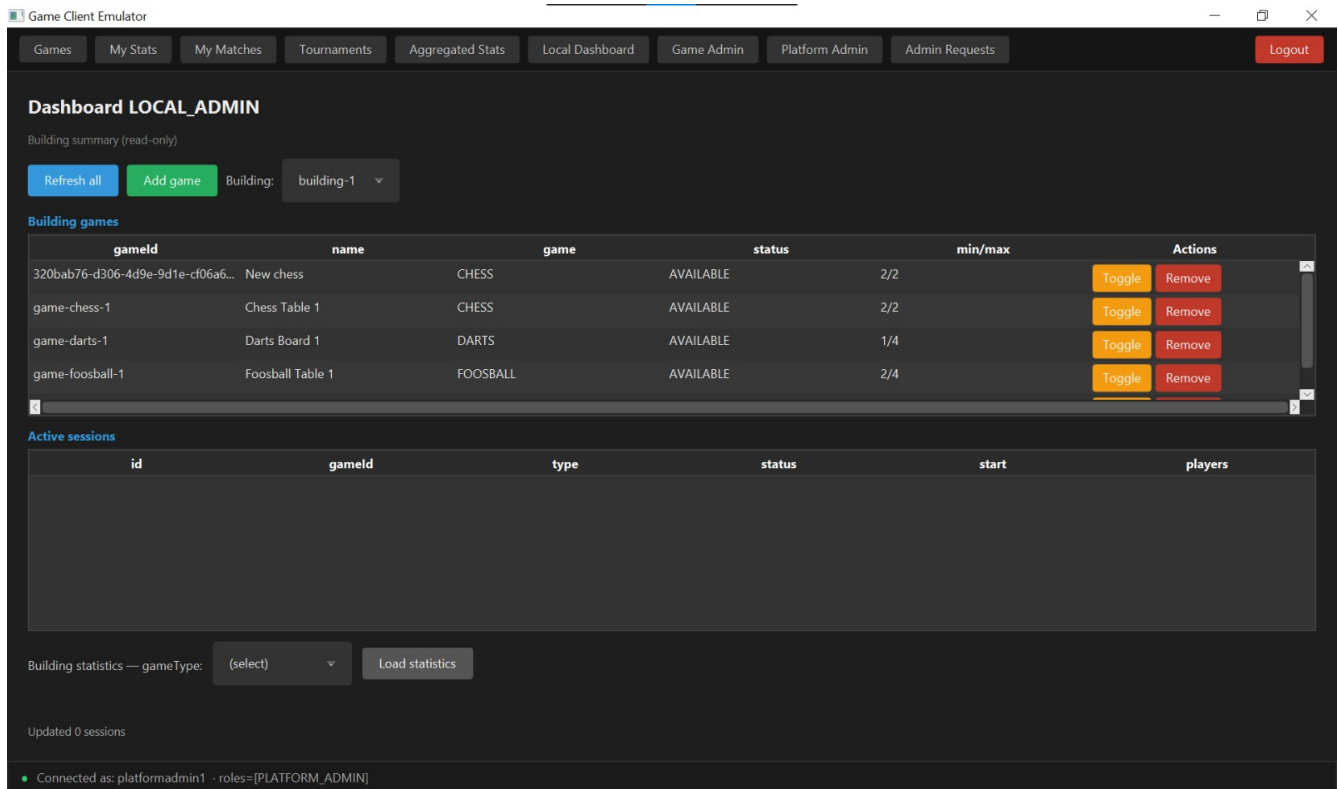
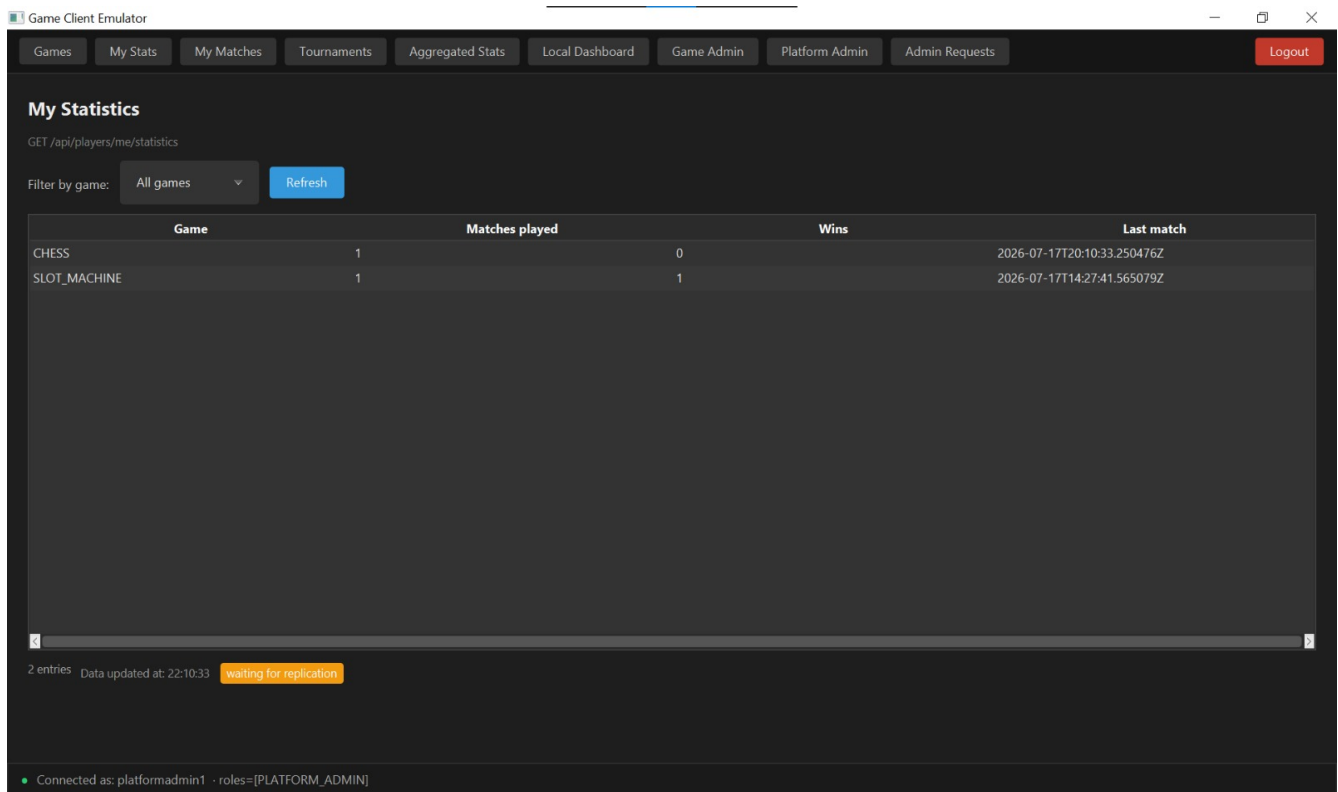


Figure 1: local_admin_dashboard.jpeg



- Il local admin ha accesso a tutti i giochi nel suo locale, con indicato se sono disponibili e i punteggi delle partite.

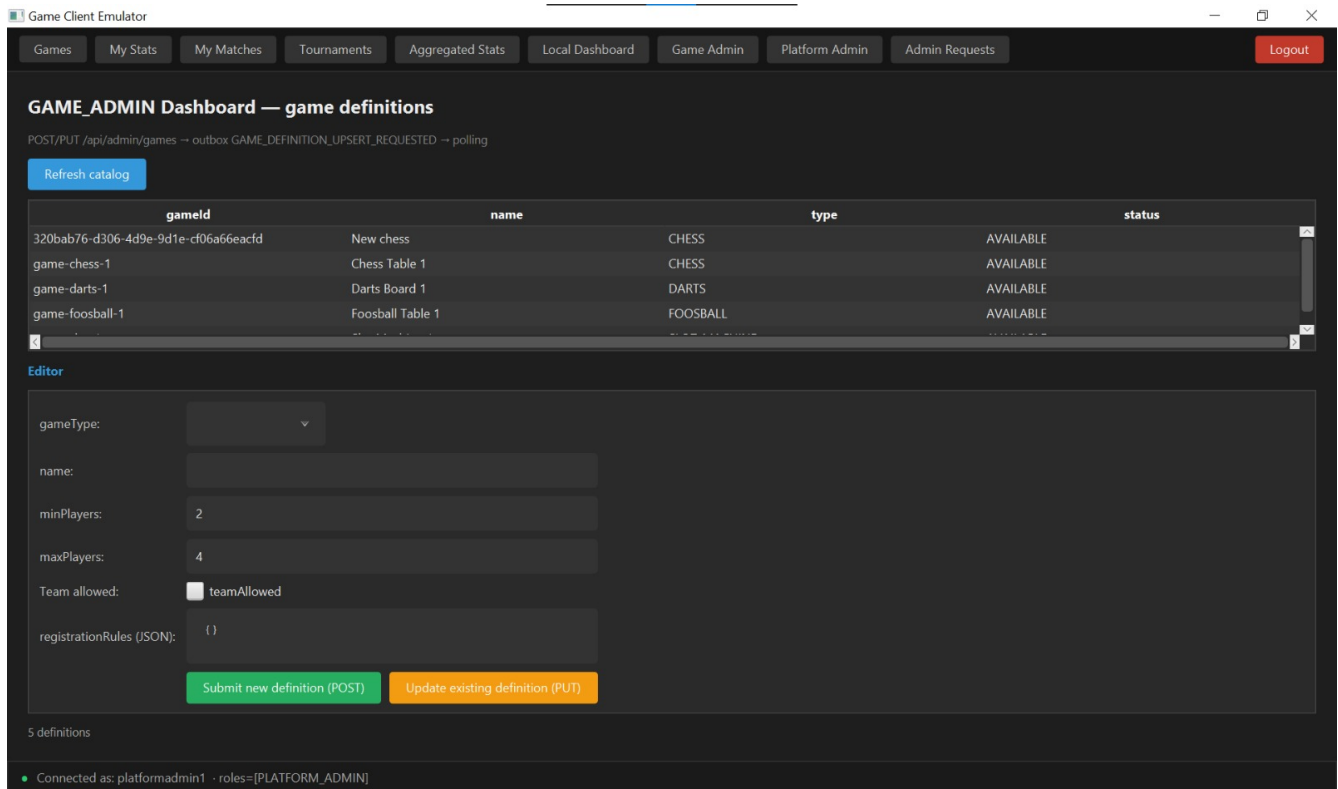


Figure 2: game_admin_dashboard.jpeg

- Il Game Admin ha accesso a tutti i giochi creati, la sua dashboard è un editor che permette di definire nuovi giochi.
- Il Platform Admin ha accesso a tutte le statistiche.

Componenti accessori

Come già accennato è necessario autenticarsi, è inoltre presente un sistema di prenotazione.

5. FASI DI LAVORO

5.1 Specifica

Come già detto l'applicazione è una piattaforma software per la gestione di sale giochi da tavolo/bar disposte in più edifici fisici.

Casi d'uso

- 1) **Prenotazione e gioco** un giocatore autenticato prenota una postazione di gioco libera nel proprio edificio; se la prenotazione non viene utilizzata entro l'orario previsto la macchina viene rilasciata automaticamente. Il giocatore avvia poi la sessione dal client, può metterla in pausa e riprenderla, e alla fine invia il risultato della partita (vincitore, punteggio, esito).
- 2) **Login e registrazione offline** un utente si autentica o crea un nuovo account anche quando il Local Server del proprio edificio è isolato dal Central System, grazie alla replica locale delle credenziali e alla firma dei token JWT con una chiave del Local Server stesso.
- 3) **Monitoraggio degli endpoint e recupero da crash** il Local Server verifica periodicamente che le postazioni di gioco connesse siano raggiungibili; se una postazione non risponde per più cicli

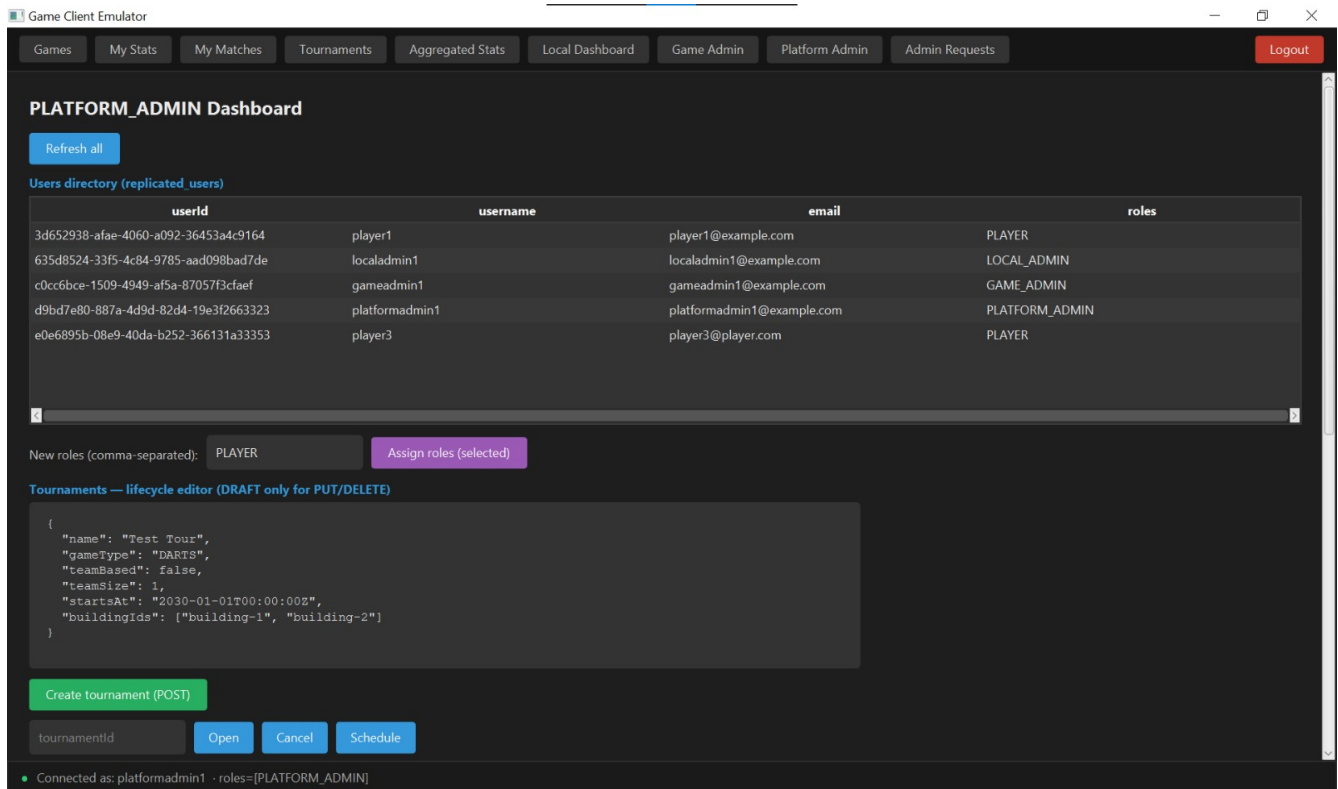


Figure 3: platform_admin_dashboard.jpeg

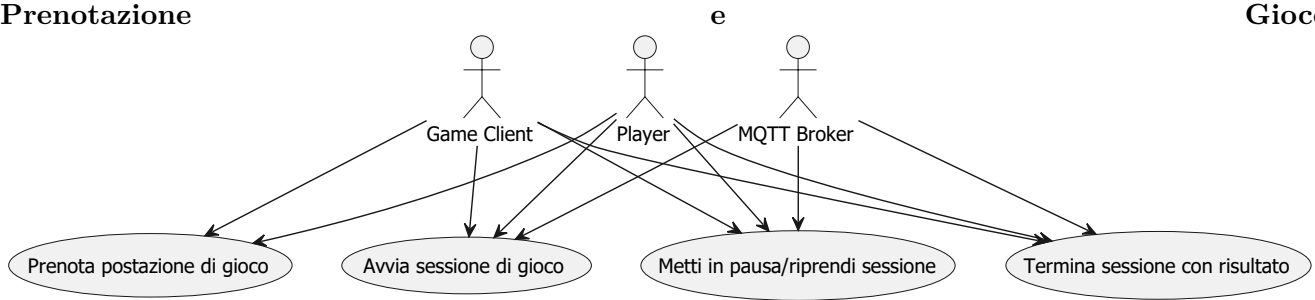
consecutivi, o se il server si riavvia dopo un crash con sessioni rimaste appese, la partita in corso viene chiusa automaticamente e la macchina torna disponibile.

- 4) **Creazione e gestione di un torneo** un Platform Admin crea un torneo (individuale o a squadre) su almeno due edifici per uno stesso gioco; i giocatori si iscrivono in autonomia durante la fase di registrazione aperta, viene generato un bracket ad eliminazione diretta e i risultati dei singoli match aggiornano automaticamente il bracket e la classifica.
- 5) **Sincronizzazione locale-centrale** tutti gli eventi generati mentre il Local Server è offline (prenotazioni, sessioni di gioco, iscrizioni) vengono accumulati in una coda locale e inviati in blocco al Central System non appena la connettività torna disponibile.
- 6) **Consultazione statistiche per ruolo** ogni tipologia di utente visualizza dati differenti: il player le proprie partite e statistiche personali, il local admin lo stato dei giochi e le statistiche del proprio locale, il game admin l'anagrafica dei giochi definiti, il platform admin le statistiche aggregate dell'intera piattaforma.
- 7) **Definizione di nuove tipologie di gioco** un Game Admin definisce nuovi tipi di gioco e le relative regole di registrazione delle partite, incluse eventuali varianti a giocatore singolo basate su esito casuale (es. slot machine, roulette).
- 8) **Acquisizione eventi dai sensori di gioco** i sensori posizionati su ciascun gioco fisico (gestiti ad esempio da una board ESP32) rilevano gli eventi significativi della partita e li inviano al Local Server tramite chiamate HTTP e MQTT.

Diagramma UML dei Casi d'Uso

Prenotazione

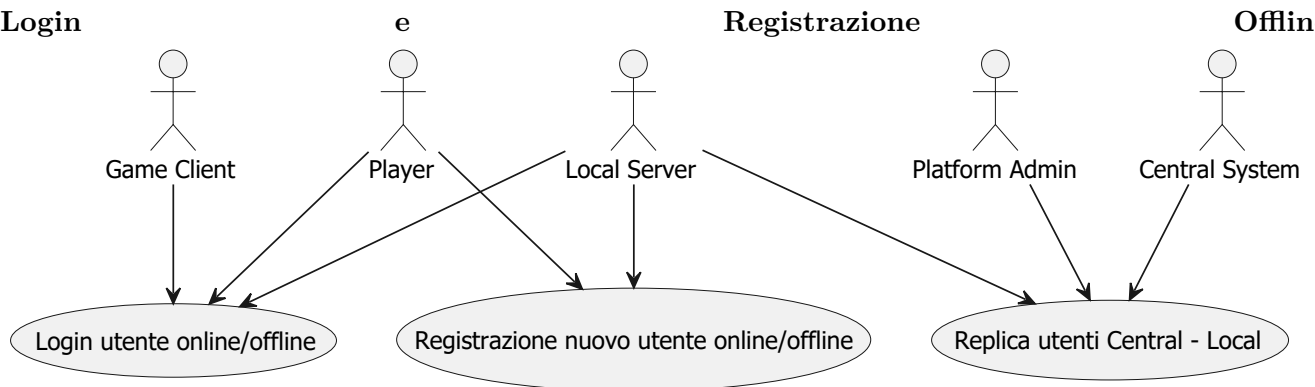
Gioco



Login

Registrazione

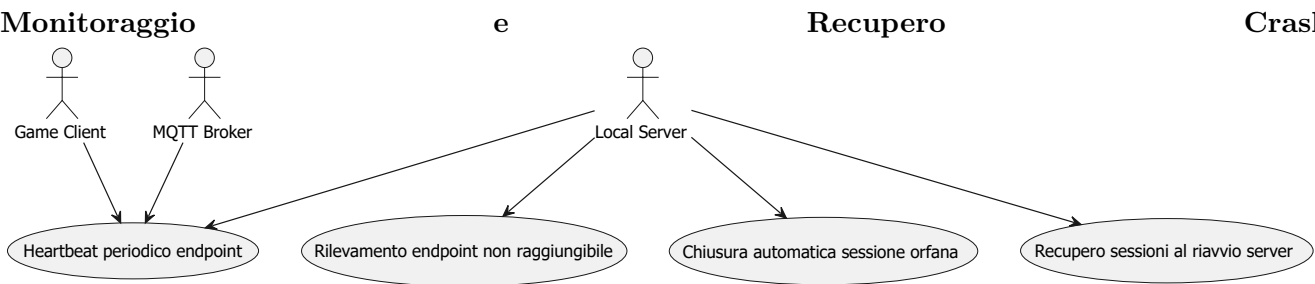
Offline



Monitoraggio

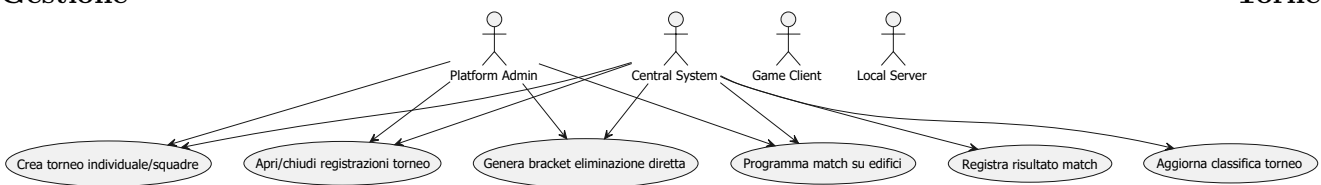
Recupero

Crash



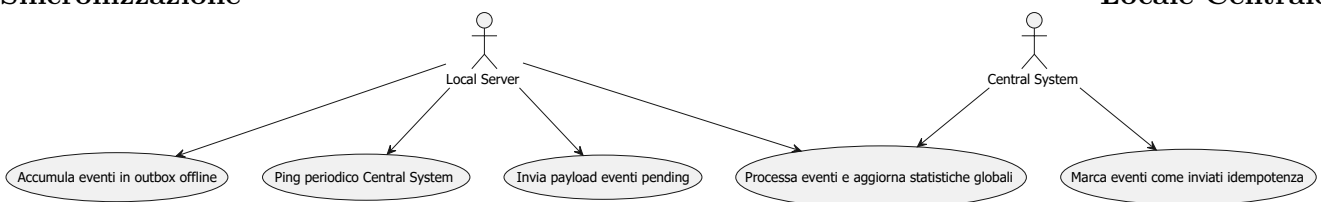
Gestione

Tornei

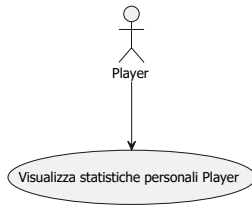


Sincronizzazione

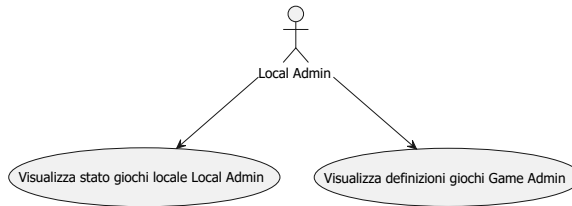
Locale-Centrale



Consultazione



Statistiche

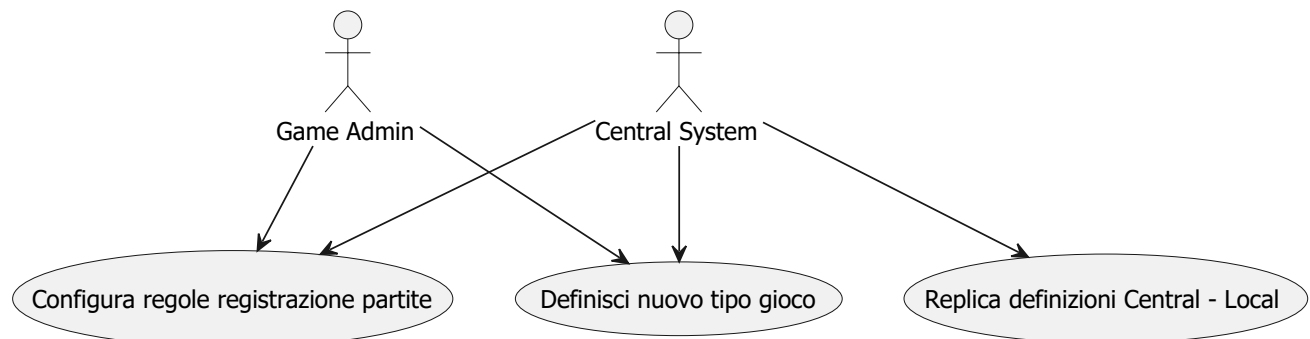


Definizione

Tipi

di

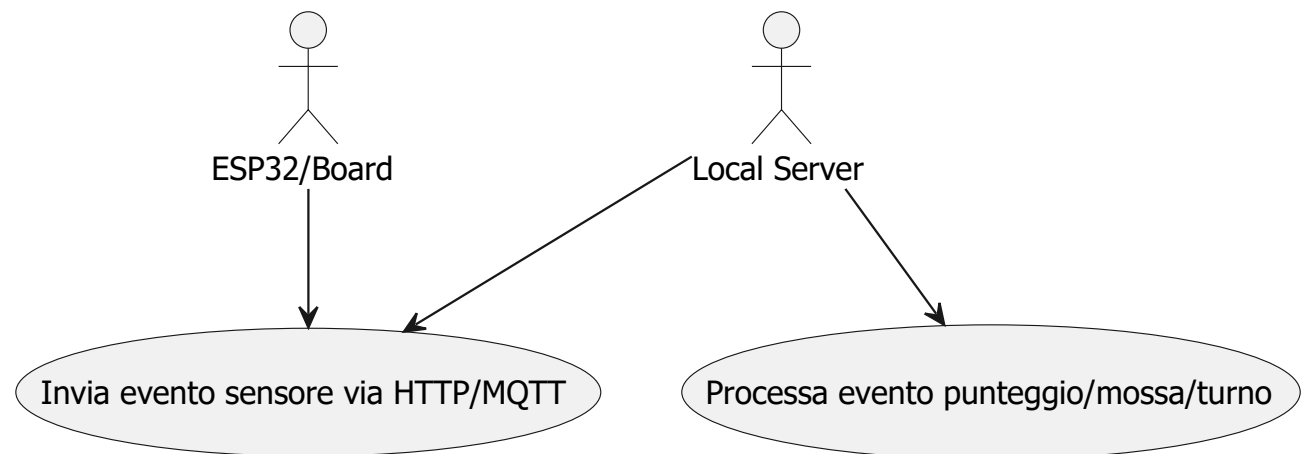
Gioco



Acquisizione

Eventi

Sensori



5.2 Progettazione

Vengono usati pattern diversi:

- Sul piano della distribuzione usiamo Hub-and-Spoke unito a quello Pub/Sub. Qui il Central System è l'hub e gli spoke i Local Server, quello Pub/Sub è attuato tramite MQTT tra server locale e Game Client.
- Sul piano dei microservizi il sistema è composto da tre microservizi Spring Boot indipendenti (Central System, Local Server, Game Client Emulator), organizzati come monorepo Maven multi-modulo con moduli condivisi (shared-domain, shared-dto, shared-mqtt) che non dipendono da nessun framework, per evitare duplicazione di codice tra i tre servizi.
- Sul piano del codice interno a ciascun microservizio, viene applicata la Clean Architecture / architettura esagonale (Ports and Adapters): il dominio (le entità e la logica di business) è Java puro, senza dipendenze da Spring o JPA, mentre l'accesso a database, MQTT e REST avviene tramite

adapter separati. Questo rispetta il Dependency Inversion Principle e rende il dominio testabile senza framework.

In sintesi, l'architettura si può riassumere così: microservizi distribuiti in pattern hub-and-spoke con comunicazione ibrida REST/MQTT, ciascun servizio internamente strutturato secondo architettura esagonale, con sincronizzazione asincrona basata su outbox pattern per garantire resilienza offline.

Diagramma dei Package (Maven Modules + Clean Architecture Layers)

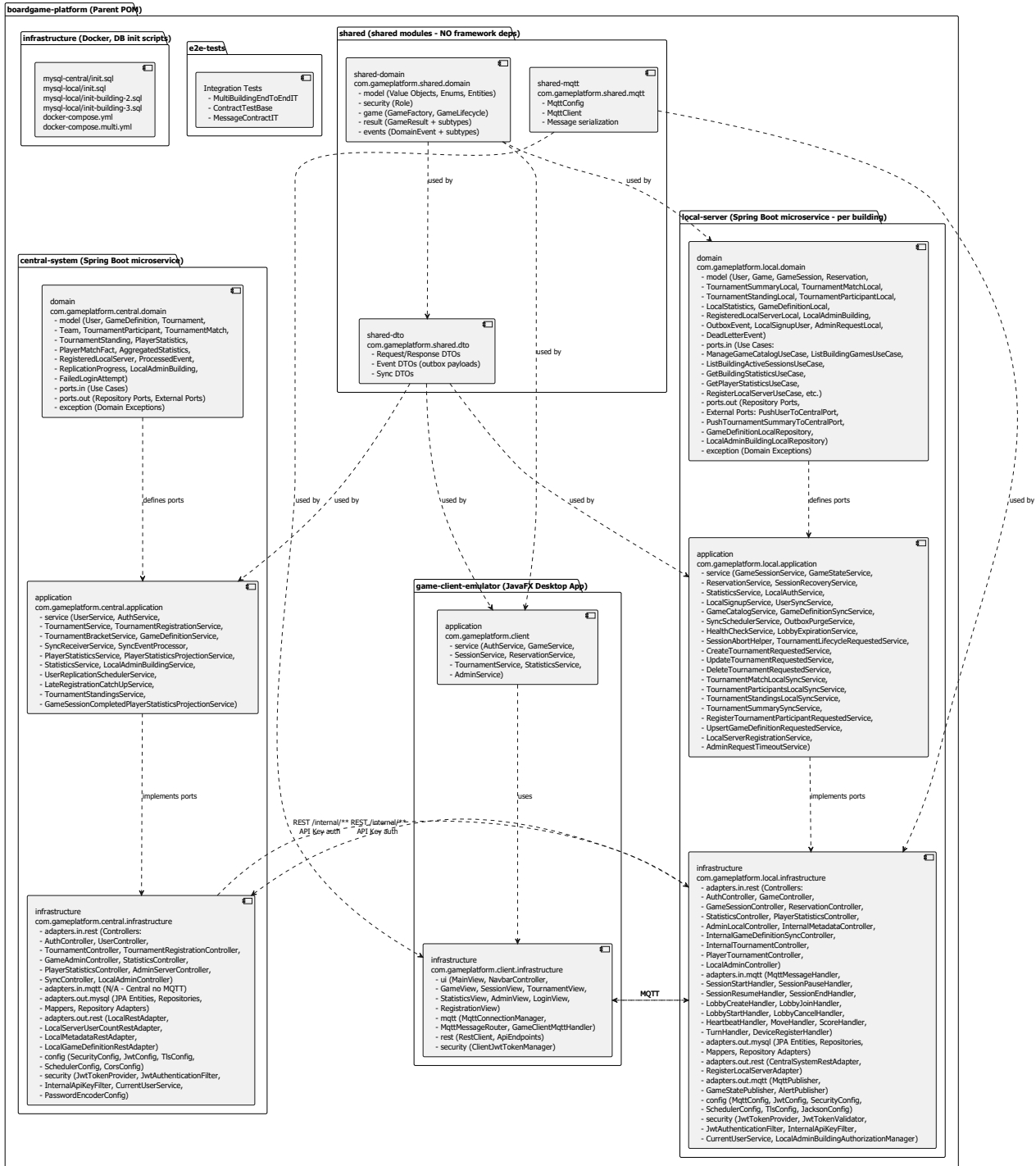


Diagramma delle Classi di Implementazione (Clean Architecture / Hexagonal)

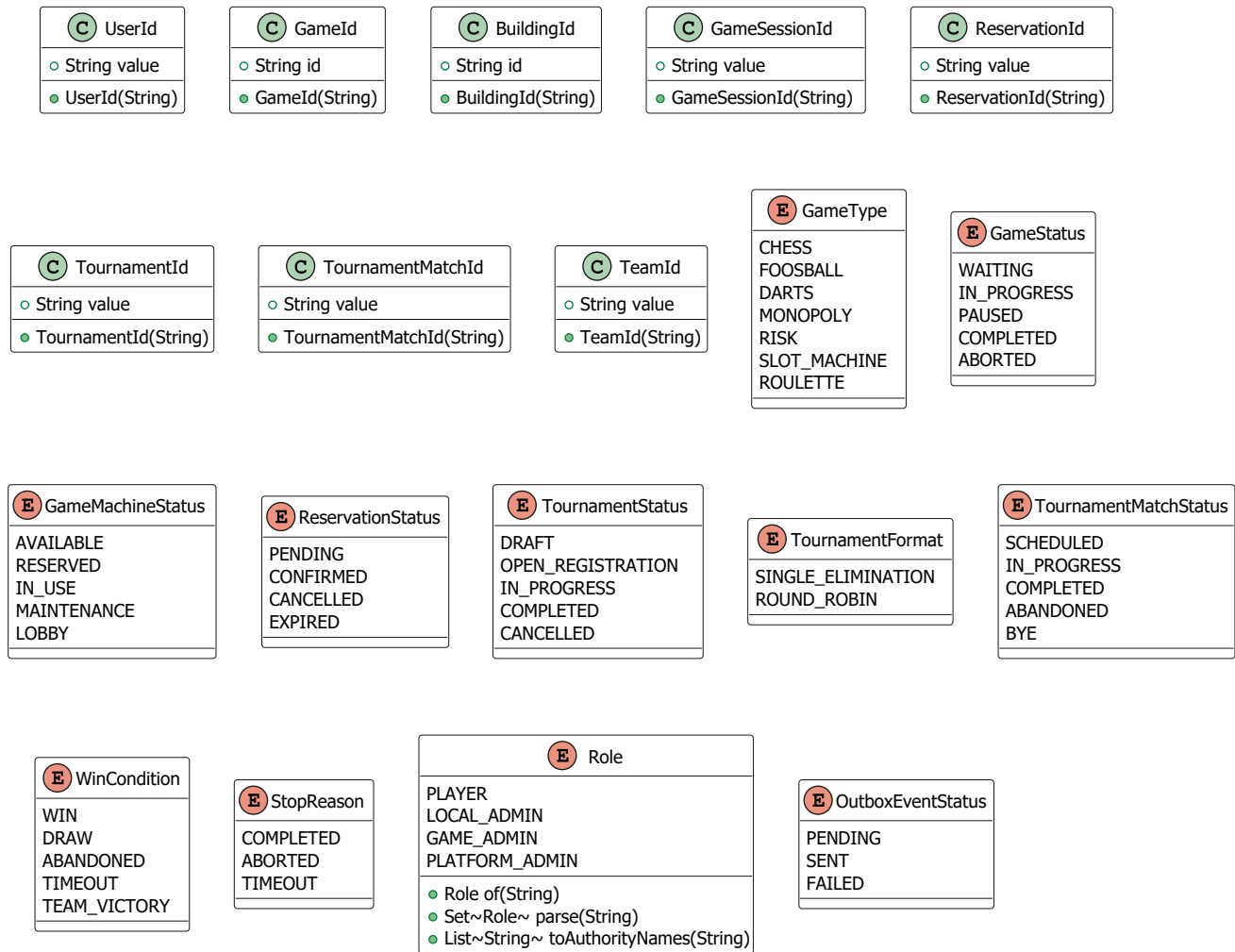
6.1

Value

Objects

&

Enum



6.2

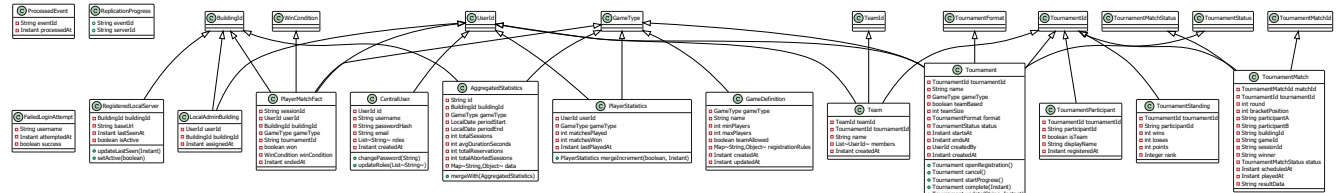
Central

System

-

Domain

Model



6.3

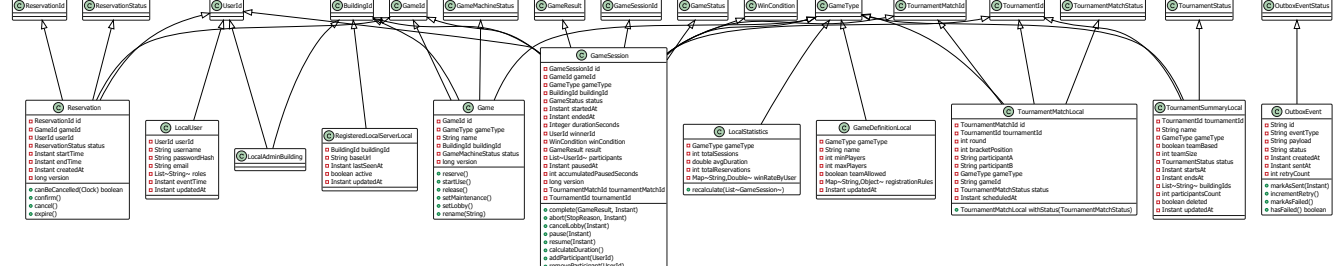
Local

Server

-

Domain

Model



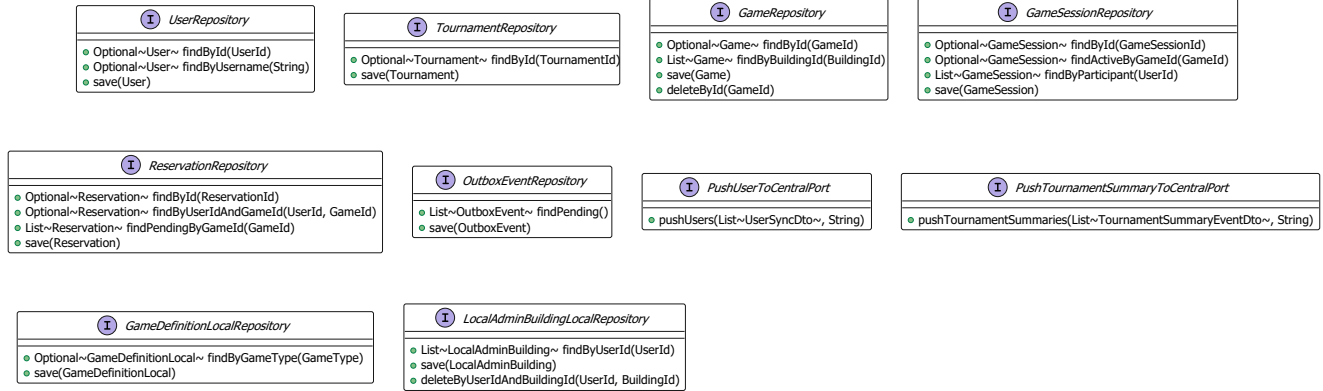
6.4

Ports

(Interfaces

/

Hexagonal)



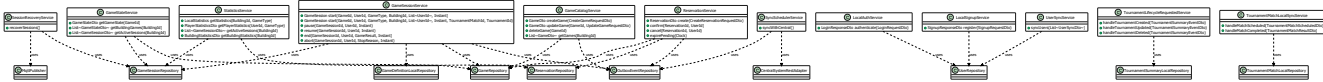
6.5

Application

Services

(Local

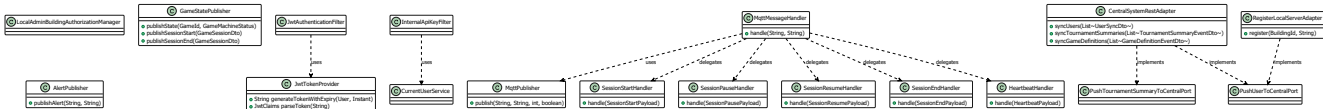
Server)



6.6

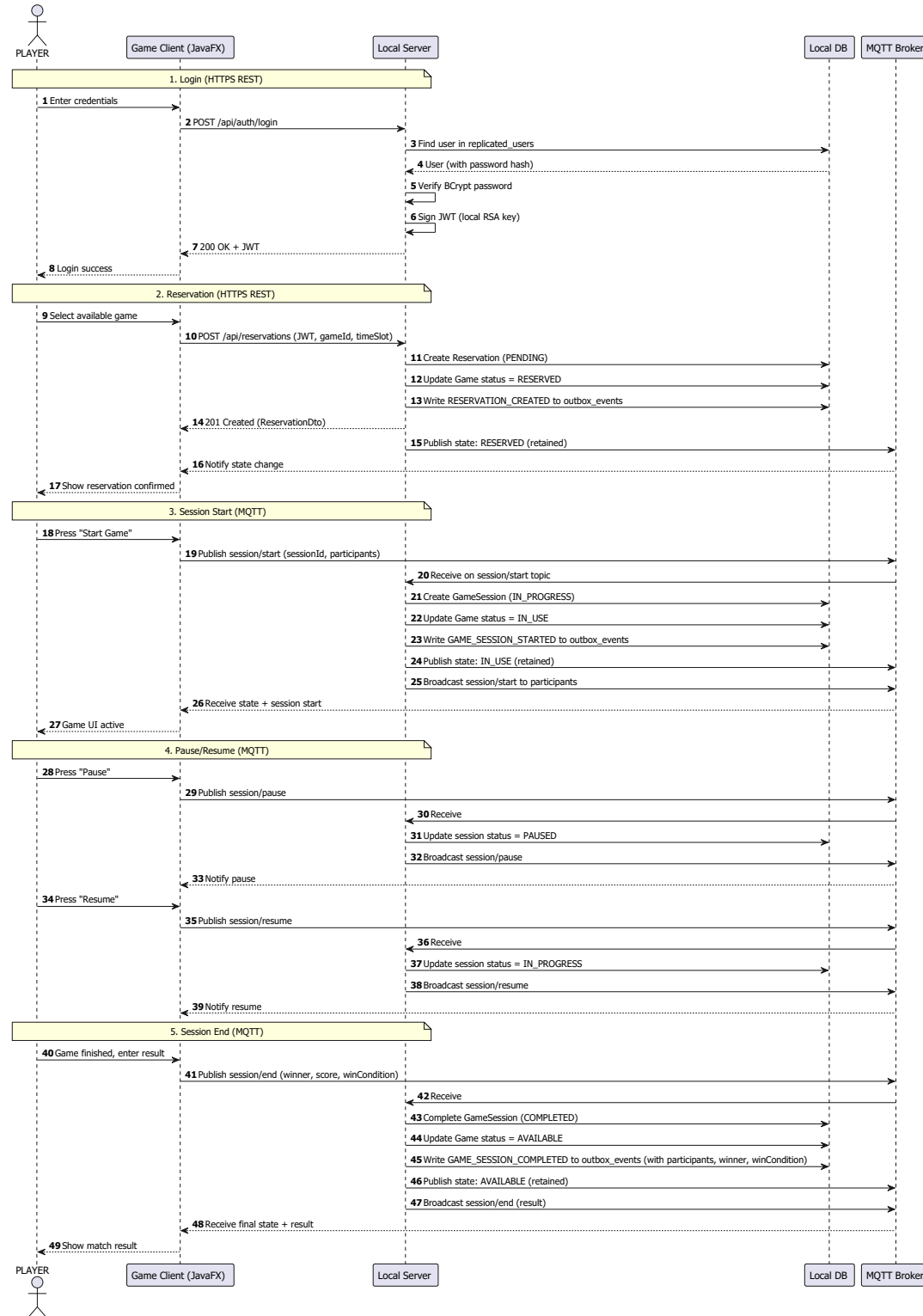
Infrastructure

Adapters

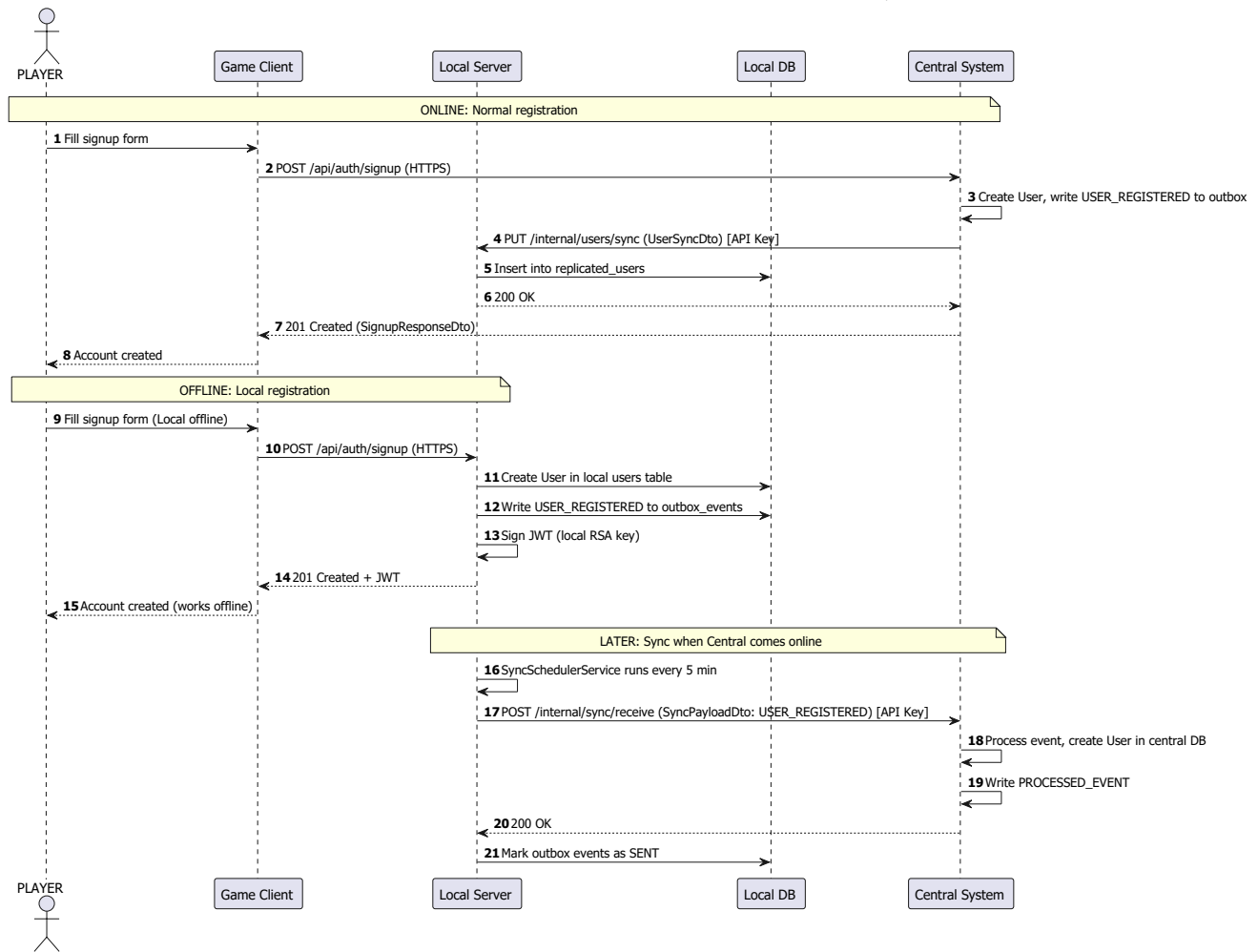


Diagrammi di Sequenza

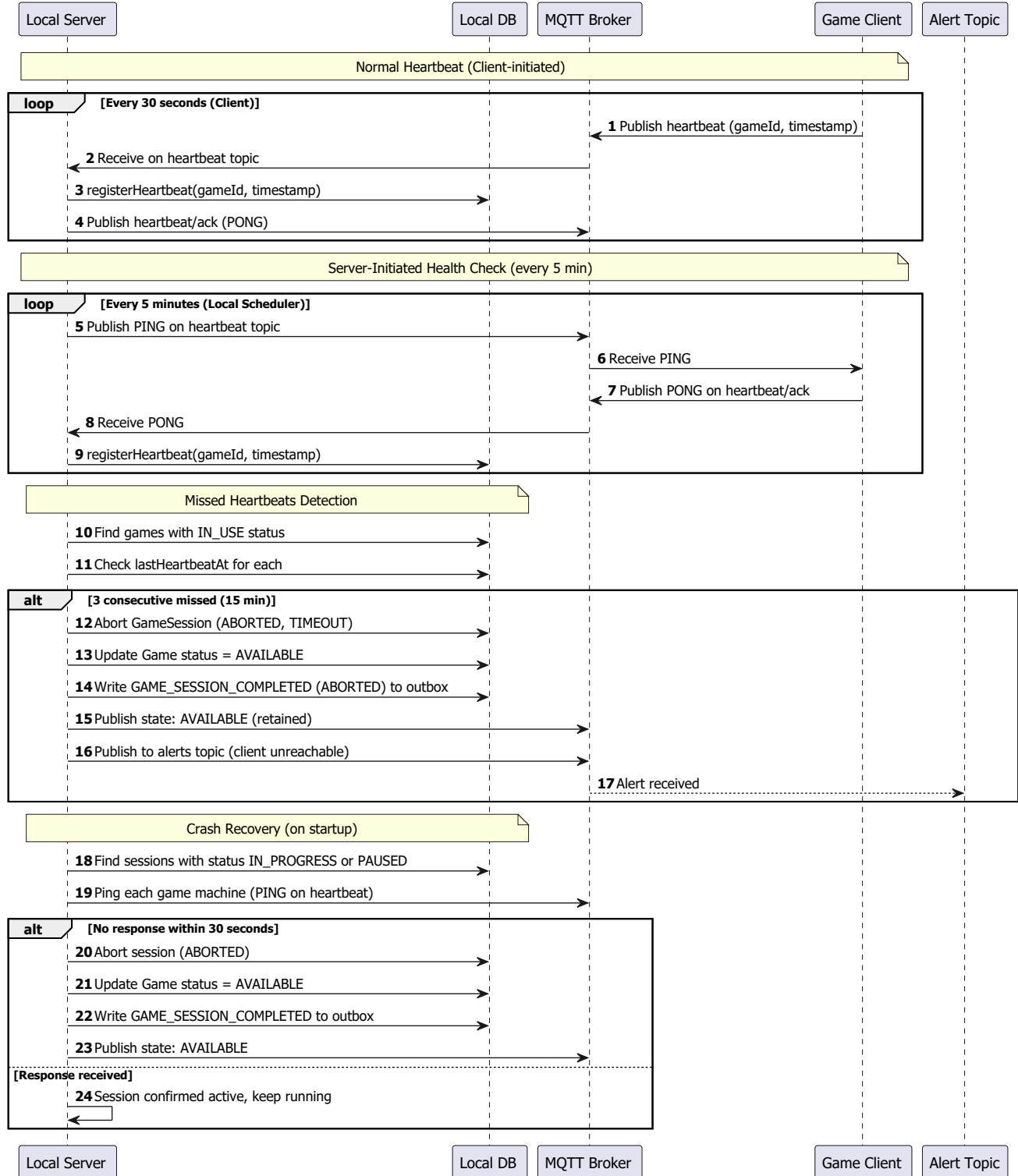
1. Prenotazione e Gioco (Reservation & Game Session)



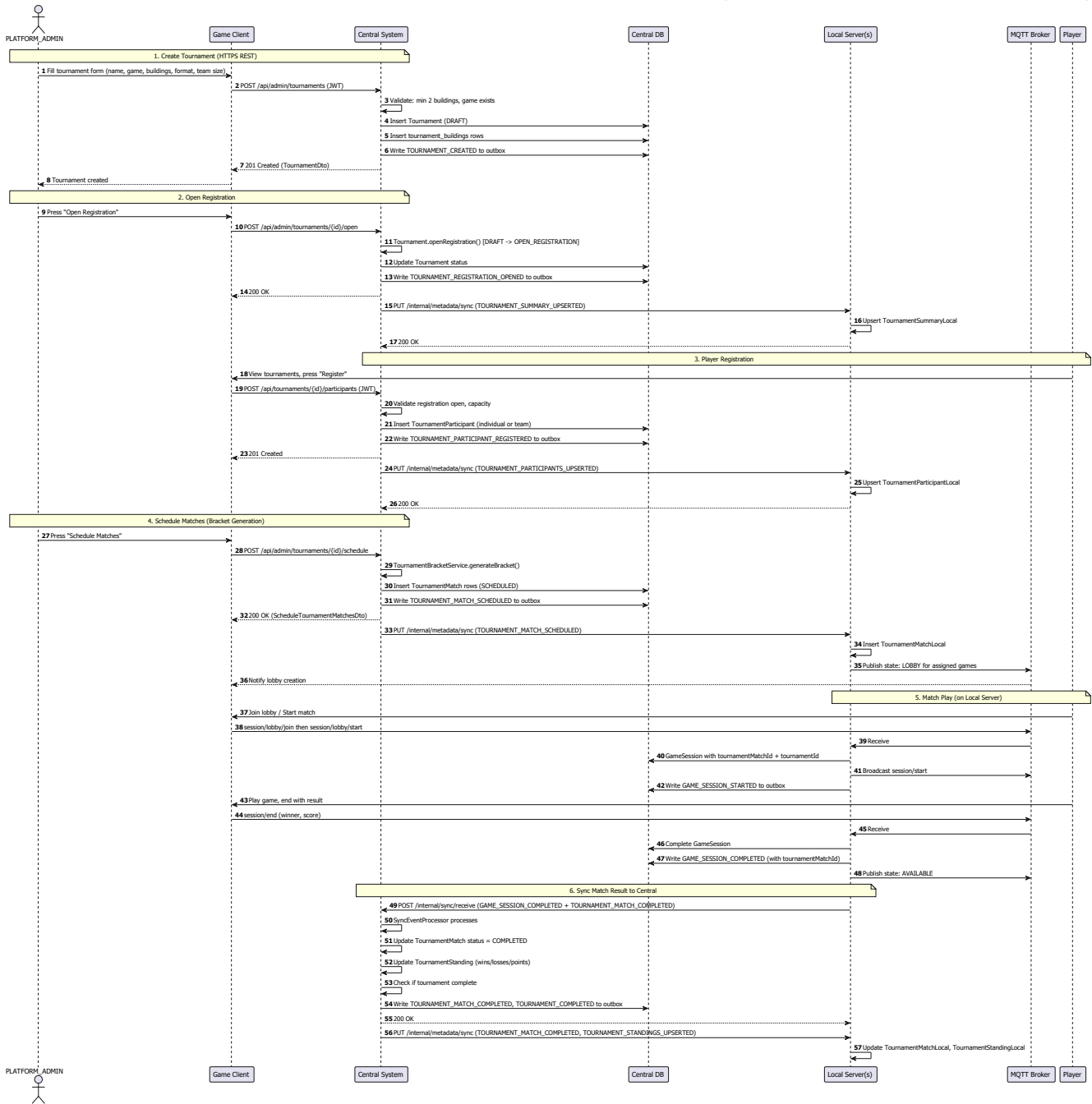
2. Login e Registrazione Offline (Offline Authentication)



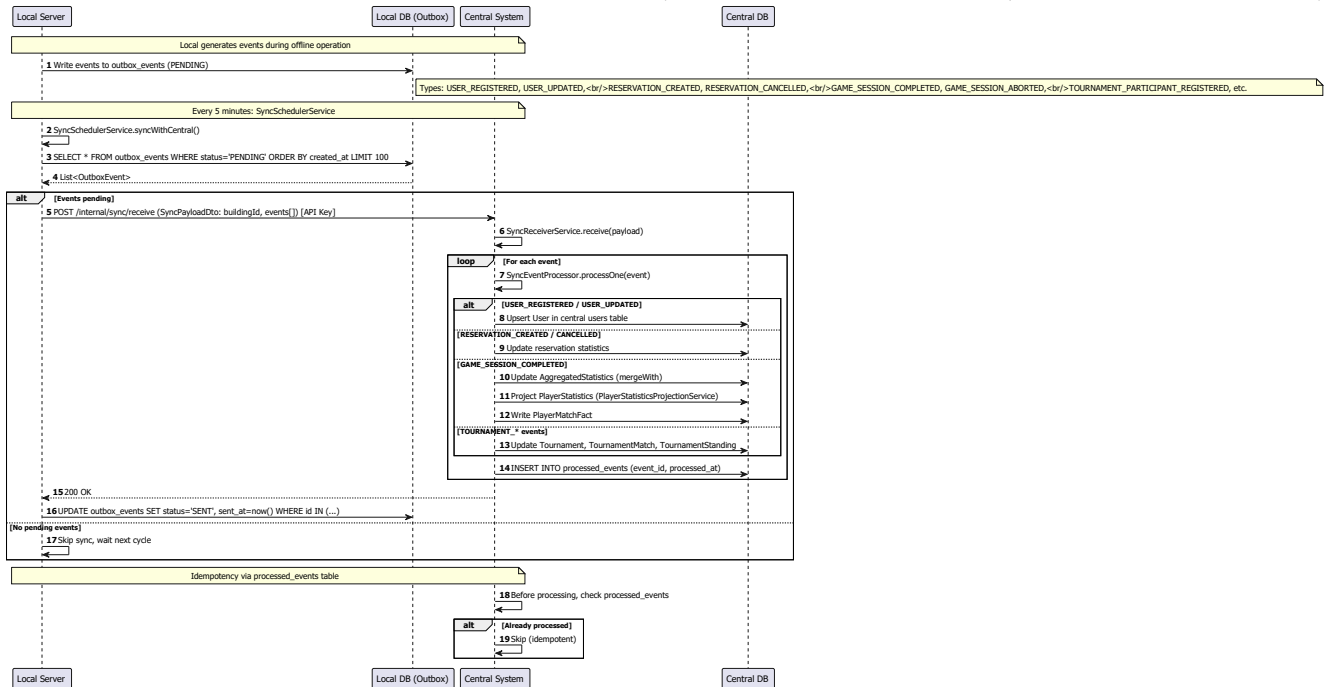
3. Monitoraggio Endpoint e Recupero da Crash (Health Check & Crash Recovery)



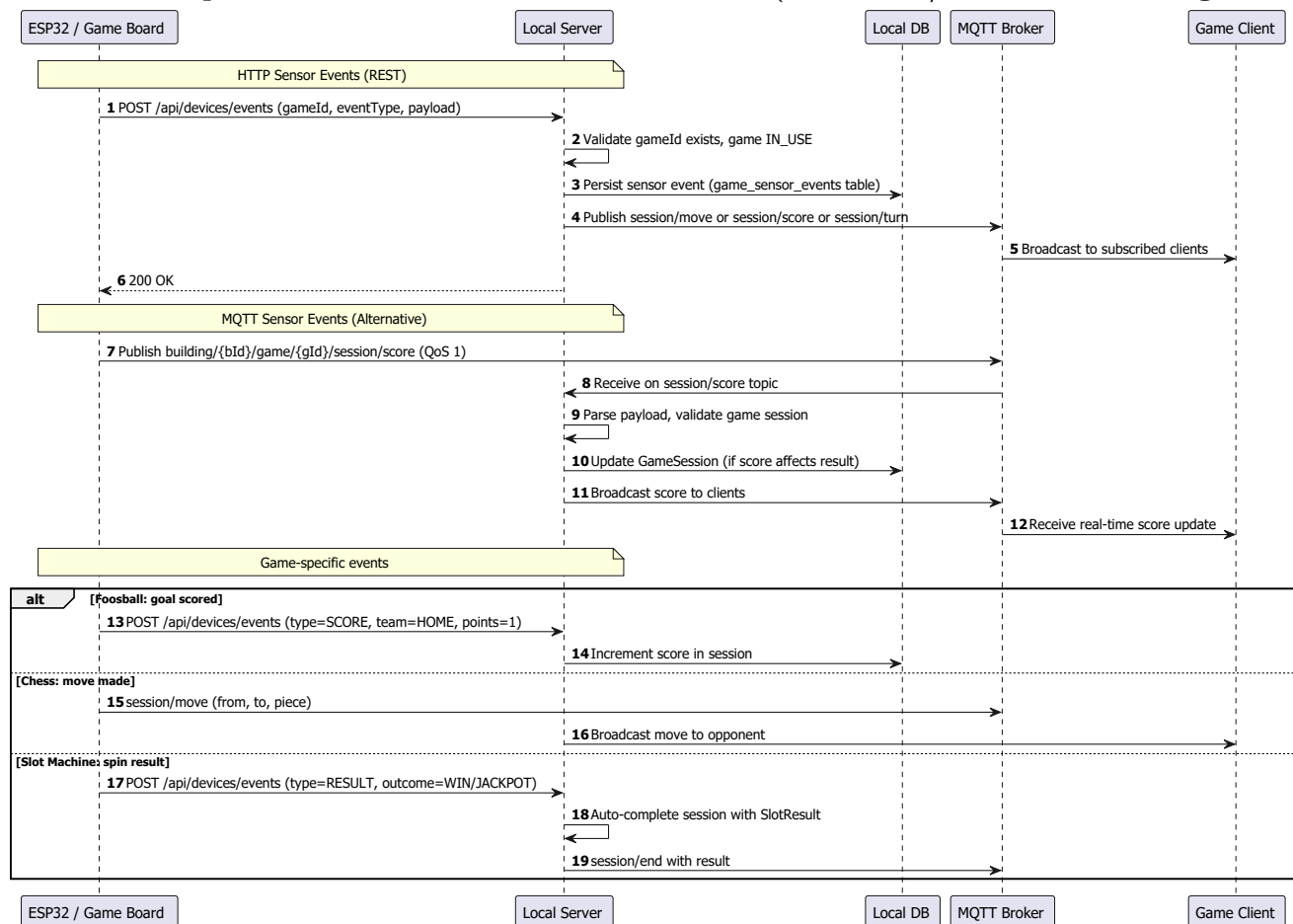
4. Creazione e Gestione Torneo (Tournament Management)



5. Sincronizzazione Locale-Centrale (Local-Central Sync / Outbox Pattern)



6. Acquisizione Eventi dai Sensori (ESP32 / Sensor Integration)



DEFINIZIONE API REST La piattaforma espone API REST da due micro servizi distinti: il Central System (porta 8180) e il Local Server (porta 8181, uno per edificio), entrambi su HTTPS/TLS 1.3.

L'accesso a ogni endpoint è protetto da autenticazione JWT (firmata con chiavi RSA proprie di ciascun nodo, non intercambiabili tra Central e Local) e regolato tramite RBAC sui quattro ruoli PLAYER, LOCAL_ADMIN, GAME_ADMIN, PLATFORM_ADMIN, con controlli aggiuntivi di self-check sull'utente e di binding sull'edificio dove previsto. Gli endpoint /internal/**, usati solo per la sincronizzazione server-to-server tra Local e Central, sono invece protetti da una API Key condivisa e non richiedono JWT.

La documentazione completa di ogni endpoint è consultabile nel documento [report_api_rest.md](#)

TOPIC MQTT Tutti i topic seguono lo schema gerarchico `building/{buildingId}/game/{gameId}/{action}`, ad eccezione di `alerts` che è a livello di edificio (`building/{buildingId}/alerts`, senza `gameId`).

Topic	Publisher	Subscriber	QoS / Retained	Descrizione
<code>building/{buildingId}/game/{gameId}/state</code>	Local Server	Game Client (wildcard +)	QoS 1, Retained	Stato della macchina di gioco: AVAILABLE, RESERVED, LOBBY, IN_USE
<code>building/{buildingId}/game/{gameId}/session/start</code>	Game Client	Local Server	QoS 1	Avvio sessione di gioco (walk-in, con <code>reservationId</code> opzionale)
<code>building/{buildingId}/game/{gameId}/session/pause</code>	Game Client	Local Server	QoS 1	Pausa della sessione in corso
<code>building/{buildingId}/game/{gameId}/session/resume</code>	Game Client	Local Server	QoS 1	Ripresa della sessione
<code>building/{buildingId}/game/{gameId}/session/end</code>	Game Client	Local Server	QoS 1	Chiusura sessione con <code>result_data</code> (vincitore, punteggio, esito)
<code>building/{buildingId}/game/{gameId}/session/lobby/create</code>	Game Client (creator)	Local Server	QoS 1	Creazione di una lobby su un gioco
<code>building/{buildingId}/game/{gameId}/session/lobby/join</code>	Game Client (joiner)	Local Server	QoS 1	Ingresso di un giocatore in una lobby esistente

Topic	Publisher	Subscriber	QoS / Retained	Descrizione
building/ {buildingId}/ game/{gameId}/ session/lobby/ start	Game Client (creator)	Local Server	QoS 1	Avvio della partita dalla lobby
building/ {buildingId}/ game/{gameId}/ session/lobby/ cancel	Game Client (creator)	Local Server	QoS 1	Annullamento della lobby prima dell'avvio
building/ {buildingId}/ game/{gameId}/ heartbeat	Game Client / Local Server	Local Server / Game Client	QoS 0	Battito periodico del client, oppure PING del server ogni 5 minuti
building/ {buildingId}/ game/{gameId}/ heartbeat/ack	Local Server / Game Client	Game Client / Local Server	QoS 0	Risposta (ACK/PONG) al battito ricevuto
building/ {buildingId}/ game/{gameId}/ session/move	Game Client / tavolo fisico	Local Server	QoS non specificato	Evento di mossa durante la partita (generico, da standardizzare per tipo di gioco)
building/ {buildingId}/ game/{gameId}/ session/score	Game Client / tavolo fisico	Local Server	QoS non specificato	Evento di punteggio (es. goal a calciobalilla, generico)
building/ {buildingId}/ game/{gameId}/ session/turn	Game Client / tavolo fisico	Local Server	QoS non specificato	Cambio turno (generico)
building/ {buildingId}/ alerts	Local Server	Central System / dashboard	QoS non specificato	Allarmi: client irraggiungibile dopo 3 heartbeat mancati (15 min), prenotazione non valida, ecc.